

Знакомство с OpenMP

часть 3

Директива **omp for**

Новые возможности (опция **collapse**)

collapse указывает, сколько циклов во вложенном цикле должно быть свернуто в одно большое пространство итераций и разделено в соответствии с условием планировщика.

collapse (положительная целая константа)

```
void work(int i, int j) {//некий код}
```

```
void good_collapsing(int n)
{
    int i, j;
    #pragma omp parallel default(shared)
    {
        #pragma omp for collapse (2)
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++)
                work(i, j);
        }
    }
}
```

Последовательное выполнение итерации во всех связанных циклах определяет порядок итераций в «свернутом» итерационном пространстве.

Директива **omp for**

Новые возможности (опция collapse)

Возможные ошибки

```
void work(int i, int j) {//некий код}
void work_with_i(int i) {//некий код}

void error_collapsing(int n)
{
    int i, j;
    #pragma omp parallel default(shared)
    {
        #pragma omp for collapse (2)
        for (i = 0; i < n; i++) {
            work_with_i (i);           // Ошибка
            for (j = 0; j < n; j++)
                work(i, j);
        }
    }
}
```

Не допускается разрыв циклов каком-либо действием или задачей.

Необходимо строгое задания циклов друг за другом.

Директива **omp for**

Новые возможности (опция collapse)

Возможные ошибки

```
void work(int i, int j) {//некий код}

void error_collapsing(int n)
{
    int i, j;
    #pragma omp parallel default(shared)
    {
        #pragma omp for collapse (2)
        for (i = 0; i < n; i++) {
            for (j = 0; j < i; j++) // Ошибка
                work(i, j);
        }
    }
}
```

Опция **collapse** может быть использована только для распределения витков циклов с прямоугольным индексным пространством

Директива **omp for**

Новые возможности опции **reduction**

reduction с OpenMP 4.0 указывает, поддерживает редукцию с массивными переменными.

Последовательный фрагмент кода

```
int size = 10;
int *hist = new int[size]

for (int y = 0; y < height; y++)
    for (int x = 0; x < width; x++)
    {
        int val = arr[y][x];
        hist[val]++;
    }
```

Параллельный фрагмент кода

```
#pragma omp parallel for reduction(+:hist[:size])
for (int y = 0; y < height; y++)
    for (int x = 0; x < width; x++)
    {
        int val = arr[y][x];
        hist[val]++;
    }
```

Векторные вычисления

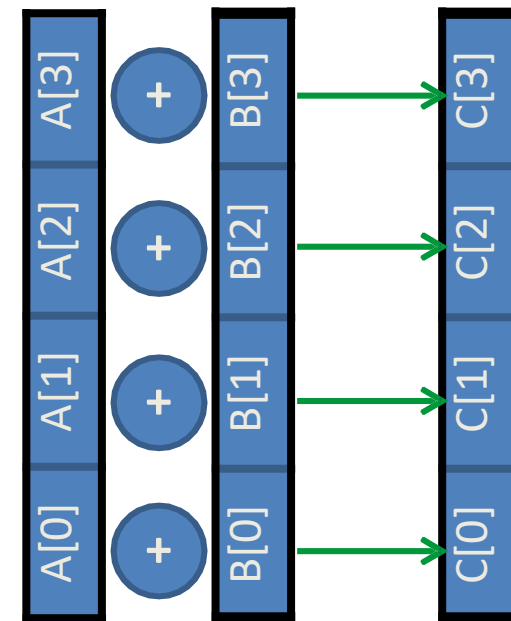
Векторные вычисления – это вид параллельных вычислений с параллелизмом на уровне данных (SIMD – Single Instruction Multiple Data)

- ❑ В современных микропроцессорах векторные вычисления поддерживаны с помощью **векторных расширений** архитектуры

Примеры векторных расширений:

MMX, SSE, AVX, ...

- ❑ Векторные расширения включают:
 - Векторные регистры – хранят множества скалярных значений
 - Векторные команды – для работы с векторными регистрами



Исполняющие блоки скалярный вариант

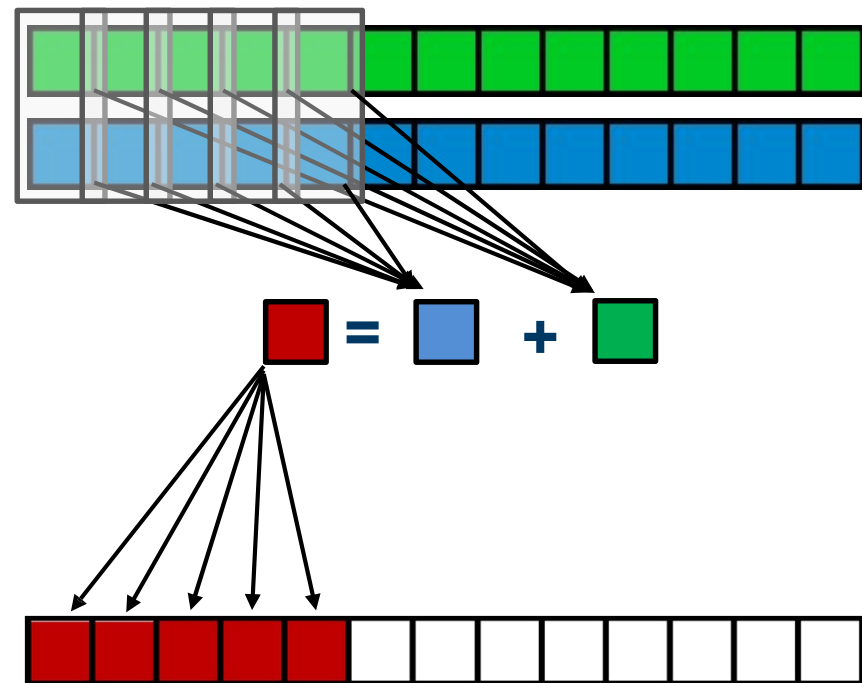
```
for (int j=0; j<size; j++){  
    A[j] = B[j] + C[j];  
}
```

Ширина регистра

- 1 operand



Скалярное исполнение



Исполняющие блоки с параллельными данными (вектор SIMD)

```
for (int j=0; j<size; j++){  
    A[j] = B[j] + C[j];  
}
```

Ширина регистра

- 1 operand



- 2 operands (SSE)



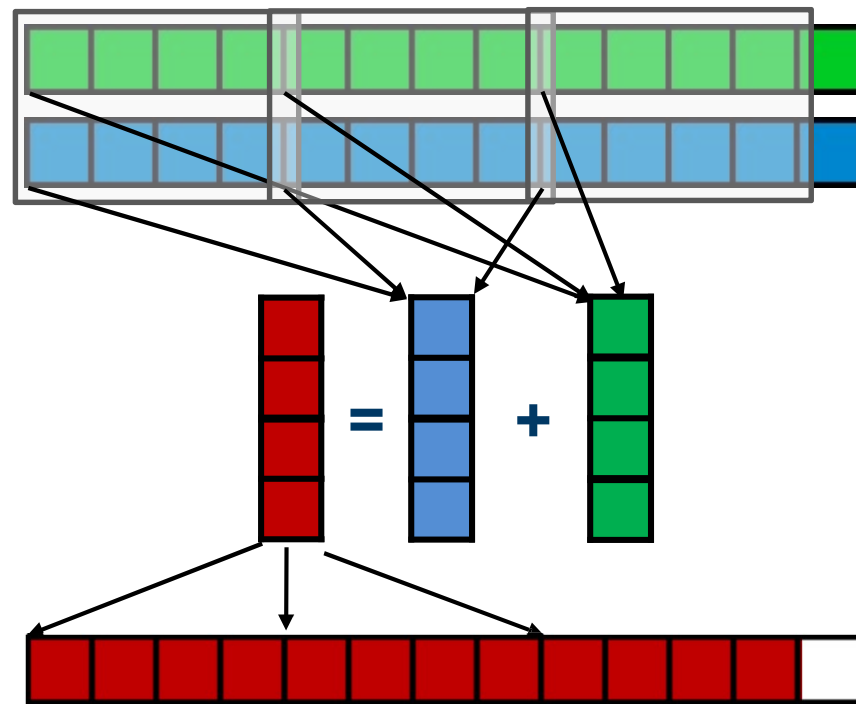
- 4 operands (AVX/AVX2)



- 8 operands (AVX512)



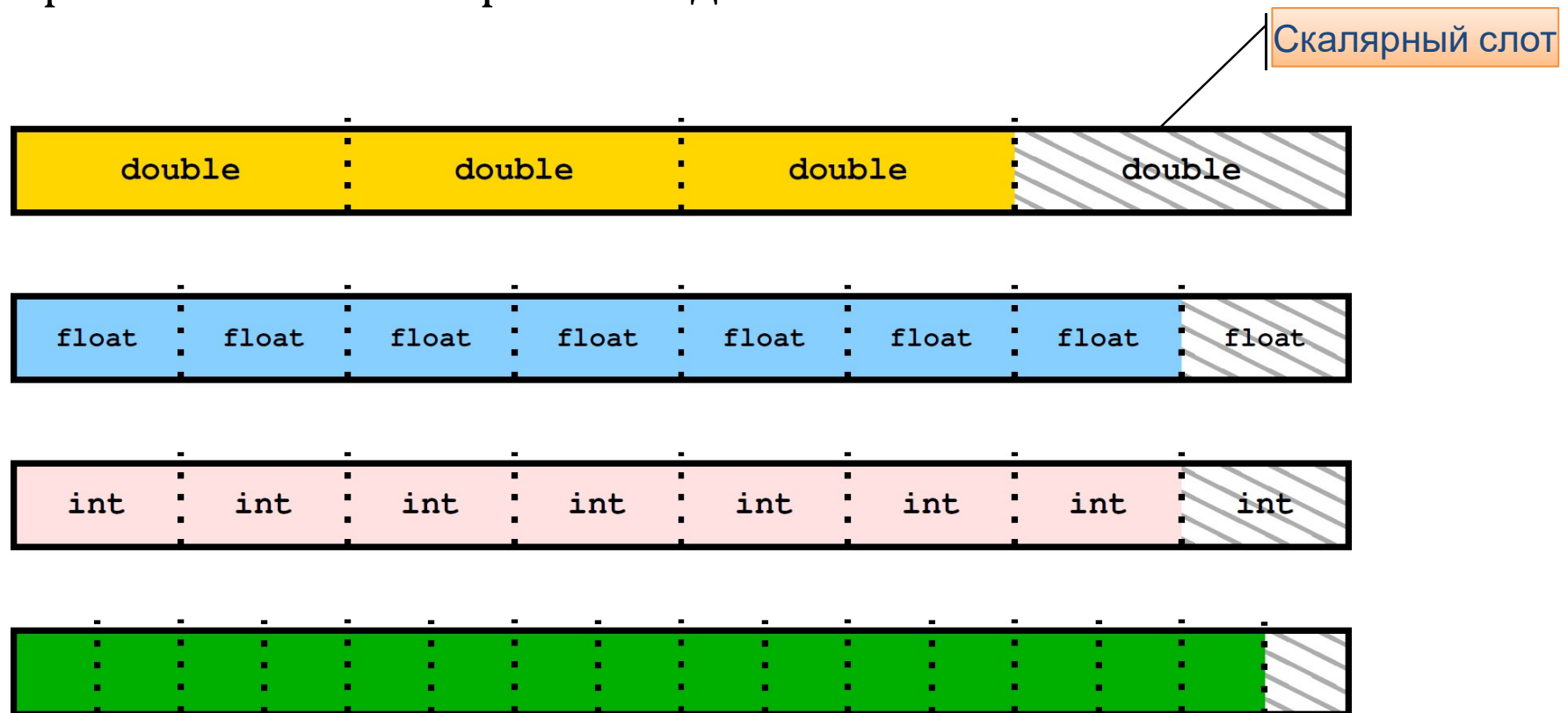
SIMD исполнение



Лучший код требует векторизации операций чтения (LOAD) записи (STORE), и арифметики.!

Типы данных в 32-байтовых SIMD-регистрах

Поддерживаемые типы данных зависят от фактического набора команд SIMD.



Логика SIMD исполнения

Шаги (выполняемые компилятором) для SIMD обработки

```
for(int i=0; i<n;i++)  
    C[i]=A[i]+B[i];
```

«Разворачивание циклов»
«Loop unrolling»



Нельзя
делать
руками

```
for(int i=0; i<n;i+=4){  
    C[i]  =A[i]  +B[i];  
    C[i+1]=A[i+1]+B[i+1];  
    C[i+2]=A[i+2]+B[i+2];  
    C[i+3]=A[i+3]+B[i+3];}  
//обработка остаточного цикла
```

Загрузка 256 бит начиная с
адреса **A[i]** в регистр **R0**

Сложение соответствующих
corresponding 64-битных записей в **R0** и **R1**
и сохранение 4 результата в **R2**

Запись **R2** (256 Bit) по адресу
начинающемуся в **C[i]**

```
LABEL1:  
→ VLOAD R0 ← A[i]  
→ VLOAD R1 ← B[i]  
→ V64ADD[R0,R1] → R2  
→ VSTORE R2 ← C[i]  
→ i←i+4  
→ i<(n-4)? JMP LABEL1  
//обработка остаточного цикла
```

Векторизация, определяемая пользователем (с OpenMP 4.0)

Директива **simd** – «просьба» компилятору векторизовать нижеследующий(-ие) цикл(-ы)

Формат директивы **simd**

#pragma omp simd *дополнительные параметры:*

safelen(length) – указывает, что никакие две итерации, выполняемые одновременно с использованием инструкций SIMD, не могут иметь расстояние в логическом пространстве итераций, превышающее **length**.

simdlen(length) – указывает предпочтительное количество итераций, которые будут выполняться одновременно. Количество итераций цикла SIMD, которые выполняются одновременно, определяется реализацией. Каждая параллельная итерация выполняется отдельной полосой SIMD.

Векторизация, определяемая пользователем (с OpenMP 4.0)

aligned (list[:alignment]) – указывает, что переменные в списке выровнены либо по указанному целочисленному значению выравнивания в единицах байтов, либо способом, зависящим от реализации.

linear (list[:linear-step]) – создает частную копию переменной, которая находится в линейной зависимости с переменной итерации цикла: $x_i = x_{\text{start}} + (i - i_{\text{start}}) * \text{linear-step}$

collapse(n) – объединить n вложенных циклов в одно итерационное пространство

Области видимости переменных

private (список_переменных)

lastprivate (список_переменных)

reduction (операция : список_переменных)

Директива simd

Пример 1

Конструкция SIMD позволяет одновременно выполнять несколько итераций связанного цикла с помощью SIMD инструкций.

```
void some_reduction_work(float* a, float* b, int n)
{
    float sum = 0.0f;
    #pragma omp simd reduction(+:sum)
    for (int k = 0; k < n; k++) {
        sum += a[k] * b[k];
    }
}
```

- Преобразует последовательное поэлементное выполнение



Директива `simd`

Пример 2

Наличие прямой зависимости в цикле, запрещает одновременное выполнение всех итераций цикла.

Опция **`safelen(4)`** указывает, что итерации цикла, находящиеся на расстоянии 4 или менее в пространстве логических итераций, могут выполняться параллельно с использованием инструкций SIMD.

```
#pragma omp simd safelen(4)
{
    for (i = 0; i < (N - 4); i++) {
        a[i] = a[i + 4] + b[i] * c[i];
    }
}
```

Правила и рекомендации для векторизации

1. Только внутренний цикл
2. Счетный цикл (количество итераций цикла может быть определено при входе в цикл)
3. У цикла один вход и один выход
4. Прямолинейный код (без условий/ветвления) если нельзя использовать маски.
5. Никаких вызовов функций (кроме встроенных математических функций и функций SIMD).

Лучшая производительность в случае:

1. Простые внутренние циклы с единичным шагом (непрерывный доступ к данным, т.е. элементы рядом друг с другом)
2. Минимизируйте косвенную адресацию (indirect addressing)
3. Выводить структуры данных по границе ширины SIMD (незначительное влияние)

Распараллеливание и векторизация ЦИКЛОВ

- ❑ Итерационное пространство циклов делится по потокам (на блоки)
- ❑ Последующее деления блоков цикла для обработки в регистрах SIMD

```
void some_reduction_work(float* a, float* b, int n)
{
    float sum = 0.0f;
    #pragma omp for simd reduction(+:sum)
    for (int k = 0; k < n; k++) {
        sum += a[k] * b[k];
    }
}
```

